

Contribution Technique Sliding Window

① Subarray - contiguous part of array

$$N=5 \quad \langle 2, 6, -1, 7, 8 \rangle \quad \text{cnt} = (5 \times 6) / 2 = 15$$

② $\langle 2, 4, 1, 6, -3, 7, 8, 4 \rangle$

③ No. of subarrays in array of
size $N = \frac{N(N+1)}{2}$

2. Given an array of integers, find total sum of all possible subarrays.

$A = [3 \quad 2 \quad 5]$

ans $\rightarrow 32$

		sum	Add all subarray sums
[3]	3	3 +	
[3 2]	3 + 2	5 +	
[3 2 5]	3 + 2 + 5	10 +	3(3) + 2(4)
[2]	2	2 +	+ 5(3)
[2 5]	2 + 5	7 +	
[5]	5	5	= 9 + 8 + 15
		<u>32</u>	= 32

BF : Go to all subarrays and calculate their sum (iterate from s to e), add it to total sum.

int totalsum = 0

for (s = 0 ; s < n ; s++) <

for (e = s ; e < n ; e++) <

// (s e)

int sum = 0

for (int i = s ; i <= e ; i++) <

sum = sum + arr[i]

totalsum += sum

// totalsum = totalsum + sum

TC: $O(N^3)$

SC: $O(1)$

Approach: Go to all subarrays and calculate sum using pf[], add to totalsum

① pf[N] $\sim N$

② int totalsum = 0

```

for (s = 0; s < n; s++)  $\sim N^2$ 
    for (e = s; e < n; e++)  $\sim N^2$ 
        // (s e)
        if (s == 0)
            sum = pf[e]
        else
            sum = pf[e] - pf[s-1]
        totalsum += sum
    
```



A = [3 2 5]

pf = [3 5 10]

TC: $O(N + N^2) = O(N^2)$

SC: $O(N)$

↓
pf[]

s	e	sum	totalsum = 0
0	0	3	3
0	1	5	8
0	2	10	18
1	1	2	20

1 2
2 2

SC : $O(N)$ $\rightarrow O(1)$

Modify the original
arr[] to modify
pf[]

Approach 3: Go to all subarrays and
calculate sum
 $\rightarrow$ carry forward

$A[] = \langle -4, 1, 3, 2 \rangle$

s	e	sum
0	0	$A[0] = -4$
1		$A[0] + A[1] = -4 + 1 = -3$
2		$A[0] + A[1] + A[2] = 0$
3		$= 0 + A[3] = 0 + 2 = 2$

int totalSum = 0

for (s = 0; s < n; s++) {

int sum = 0

for (e = s; e < n; e++) { // s e

sum = sum + A[e]

totalSum += sum

TC : $O(N^2)$

SC : $O(1)$

$$A = \langle -4, 1, 3, 2 \rangle$$

s	e	sum = 0	total sum = 0
0	0	-4	-4
0	1	-3	-7
0	2	0	-7
0	3	2	-5
sum = 0			
1	1	1	-4
1	2	4	0
1	3	6	6
sum = 0			
2	2	3	9
2	3	5	14
sum = 0			
3	3	2	16

Approach 4: Contribution Technique

We are going to every element, get their contribution and add it to total sum.

$$\text{totalSum} = \text{contri of } 0^{\text{th}} \text{ ele} + \text{contri of } 1^{\text{st}} \text{ ele} + \dots + \text{contri of } (N-1)^{\text{th}} \text{ ele}$$

Contribution of i^{th} ele = $A[i] \times \text{no. of subarrays in which } A[i] \text{ is present}$

1. In how many subarrays element at idx 1 is present?

A : ⁰3 ¹-2 ²4 ³-1 ⁴2 ⁵6

S	e	0, 1	1, 1
0	1	0, 2	1, 2
1	2	0, 3	1, 3
	3	0, 4	1, 4
	4	0, 5	1, 5
	5		

ans = 10

$(2) \times (5) = 10 \text{ subarrays}$

2. In how many subarrays element at idx 2 is present? $(i+1) \times (N-i)$

A : ⁰3 ¹-2 ²4 ³-1 ⁴2 ⁵6

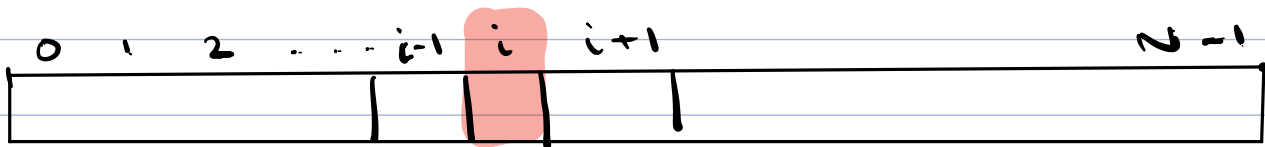
S	e
0	2
1	3
2	4

ans = 12

$$(3) \times (4) = 12 \text{ subarrays}$$

Generalized calculation

Arr size $\rightarrow N$



Starting idx
of subarray
[0 $\rightarrow$ i]

$$(i+1)$$

Ending idx of
subarray
[i $\rightarrow$ N-1]

$$(N-i)$$

$$[a \ b] = b - a + 1$$

Total subarray which contain
in idx element = $(i+1) \times (N-i)$

Contribution of in el = $A[i] \times (i+1) \times (N-i)$

int totalSum = 0

for (i = 0 ; i < n ; i++)

cnt = (i+1) * (N-i)

contri = A[i] * cnt

totalSum += contri

return totalSum

TC: $O(N)$

SC: $O(1)$

Sum of all subarrays

Sum of all
Product of all
all of all

Think about
contribution
technique

Q. Find total no. of subarrays of length k

Subarray of fixed length is called window.

Length	Start of 1st window	Start of last window	# windows
1	0	$N-1$	1
2	0	$N-2$	$N-1$
3	0	$N-3$	$N-2$
...			
k	0	$N-k$	$N-k+1$

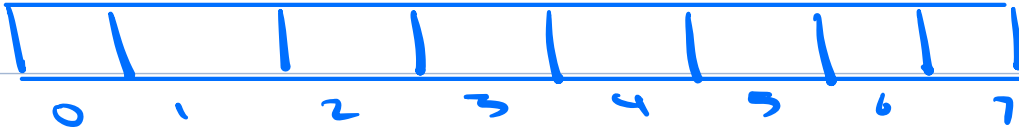
Total no. of subarrays of
len k = $N-k+1$

$N=7$ $k=4$, total no. of subarrays
of len k

$$N-k+1 = 7-4+1 = 4$$

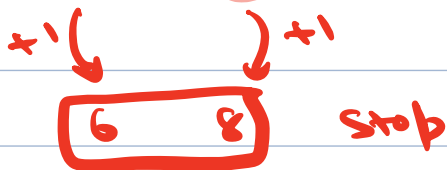
Q. Given an array of size N , print start and end indices of subarrays of length k .

$N=8$ $k=3$



s	e
0	2
1	3
2	4
3	5
4	6
5	7

s	e
0	$k-1$
1	k
2	$k+1$
3	$k+2$
⋮	⋮
—	$\underline{N-1}$

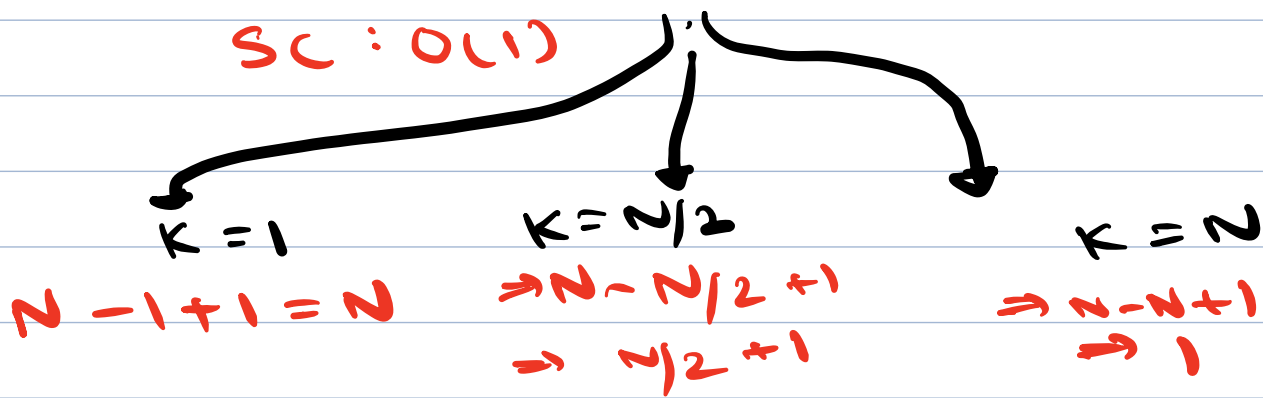


```
// N, k
int s=0, e=k-1
while (e < n) <
    print (s, e)
    s++   e++
    ↓
```

k is dependent on N

$$TC: O(N - K + 1) \approx O(N)$$

$$SC: O(1)$$



Given an array of N elements, print maximum subarray sum with length = k .

arr = $[-3, 4, -2, 5, 3, -2, 8, 2, -1, 4]$

$N = 10, k = 5$

ans = 16

s	e		sum
0	4	$(-3) + 4 + (-2) + 5 + 3$	7
1	5	$4 + (-2) + 5 + 3 + (-2)$	8
2	6	$(-2) + 5 + 3 + (-2) + 8$	12
3	7	$5 + 3 + (-2) + 8 + 2$	16
4	8	$3 + (-2) + 8 + 2 + (-1)$	10
5	9	$-2 + 8 + 2 + (-1) + 4$	11

Approach 1 : Iterate on all subarrays of len k , get sum and get max

BF

```
int maxSum = INT_MIN (-∞)
```

```
int s = 0, e = k - 1
```

```
while (e < n) < // [s e]
```

```
int sum = 0
```

```
for (i = s; i ≤ e; i++) <
```

```
sum = sum + arr[i]
```

```
maxSum = max(maxSum, sum)
```

```
s++ e++
```

1 subarray $\rightarrow O(k)$

$T.C : O((N - k + 1) \times k) = O(N^2)$

$S.C : O(1)$

$k = 1$

$(N - 1 + 1) \times 1$
 $\Rightarrow N$

$k = N/2$

$(N - N/2 + 1) \times N/2$
 $(N/2 + 1) \times N/2$
 $O(N^2)$

$k = N$

$(N - N + 1)(N)$
 $\Rightarrow N$

① Build $pf[]$ $\rightarrow N$

② $int \text{ maxSum} = INT_MIN \text{ } (-\infty)$

$int \text{ } s = 0, e = k - 1$

$while (e < n) \{ \quad // [s, e] \quad \rightarrow N - k + 1$

$if (s == 0)$

$\text{sum} = pf[e]$

$else$

$\text{sum} = pf[e] - pf[s - 1]$

$\text{maxSum} = \max(\text{maxSum}, \text{sum})$

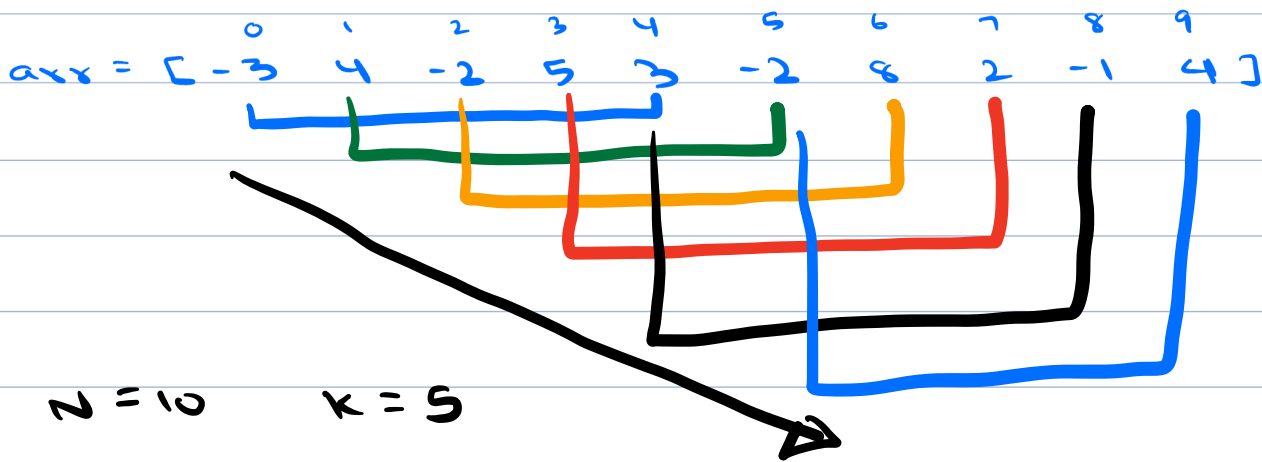
$s++ \quad e++$

$\} \quad // \text{ subarray } \rightarrow O(k)$

$TC: O(N + N - k + 1) \approx O(N)$

$SC: O(N) \rightarrow pf[]$

Approach 3: Reduce SC?



$$0 \rightarrow 4 = 7$$

$$1 \rightarrow 5 = 7 + (-2) - (-3) = 8$$

$$= \text{sum} + a[5] - a[0]$$

$$2 \rightarrow 6 = 8 + 8 - 4 = 12$$

$$= \text{sum} + a[6] - a[1]$$

$$3 \rightarrow 7 = 12 + 2 - (-2) = 16$$

$$= \text{sum} + a[7] - a[2]$$

$$4 \rightarrow 8 = 16 + (-1) - 5 = 10$$

$$= \text{sum} + a[8] - a[3]$$

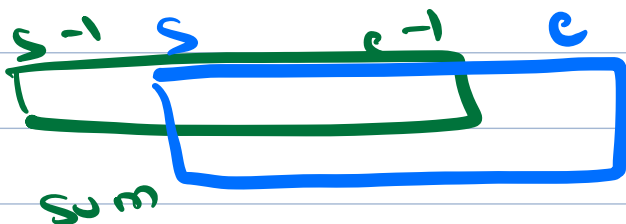
$$5 \rightarrow 9 = 10 + 4 - 3 = 11$$

$$= \text{sum} + a[9] - a[4]$$

s-1
s

e-1
e

sum
 $\text{sum} + a[e] - a[s-1]$



```
int maxSum = INT_MIN
```

```
int s = 0, e = k - 1
```

```
int sum = 0
```

```
for (i = s ; i ≤ e ; i++)
```

```
    sum = sum + a[i]
```

```
maxSum = max(maxSum, sum)
```

```
s++ e++
```

```
while (e < n) < // [s e]
```

```
    sum = sum + a[e] - a[s-1]
```

```
    maxSum = max(maxSum, sum)
```

```
    s++ e++
```

TC : $O(k + N - k) = O(N)$

SC : $O(1)$

Subarray size is fixed

